BrewLite

TÀI LIỆU YÊU CẦU NGƯỜI DÙNG (USER STORIES)

Hệ thống Đặt & Thanh toán Đồ uống Không dùng Tiền mặt (Assignment 1)

Đơn vị phát triển: Nhóm Đồ án BrewLite
Kiến trúc: Monorepo (Next.js Frontend + NestJS API + PostgreSQL/Prisma)

Ngày 14 tháng 11 năm 2026

Tóm tắt nội dung

Tài liệu này mô tả các vai trò người dùng (Personas) và danh sách User Stories kèm tiêu chí nghiệm thu (Acceptance Criteria) theo đúng phạm vi triển khai thực tế của các Task trong **Assignment 1 – BrewLite**, bao gồm: duyệt thực đơn, tùy chọn món, giỏ hàng, xác thực JWT, đặt hàng, thanh toán không tiền mặt (Mock Gateway, VNPay Sandbox), xác nhận/theo dõi đơn hàng và xử lý nghiệp vụ Backend nâng cao.

Mục lục

1 Vai trò người dùng (User Personas) 1

2 Chi tiết User Stories theo Task (Assignment 1) 2

2.1 Epic 1: Thực đơn, Tùy chọn món & Giỏ hàng (Task 2, 3, 4, 5) 2
2.2 Epic 2: Đăng ký & Đăng nhập tài khoản (Task 7 – FR-04) 5
2.3 Epic 3: Đặt hàng, Khuyến mãi & Thanh toán Không tiền mặt (Task 6, 8, 10) . 6
2.4 Epic 4: Xác nhận, Theo dõi lịch sử & Điều phối đơn hàng (Task 9, 10) 8

---

1 Vai trò người dùng (User Personas)

Mã	Vai trò	Mô tả & Phạm vi thao tác chính
CUS	Khách hàng (Customer)	Xem thực đơn, tùy chọn Size/Topping, quản lý giỏ hàng, đăng ký/đăng nhập (JWT), áp mã khuyến mãi, thanh toán không tiền mặt, xem xác nhận và lịch sử trạng thái đơn hàng.
BAR	Barista / Quản lý quầy	Truy cập trang điều phối (`/admin`) để theo dõi danh sách đơn hàng theo trạng thái (làm mới định kỳ 5 giây) và cập nhật tiến trình pha chế theo Order State Machine.

2 Chi tiết User Stories theo Task (Assignment 1)

2.1 Epic 1: Thực đơn, Tùy chọn món & Giỏ hàng (Task 2, 3, 4, 5)

US-01: Duyệt menu, lọc danh mục, tìm kiếm và xử lý trạng thái tải/lỗi

Mô tả (Story Narrative):

> Là một **Khách hàng (Customer)**,
> Tôi muốn **xem danh sách thức uống từ API, lọc theo danh mục, tìm kiếm theo tên và nhận thông báo rõ ràng khi tải hoặc lỗi**,
> Để **dễ dàng chọn được đồ uống còn hàng trong menu**.

Thông tin bổ sung:

- **Task liên quan:** Task 2 (`GET /products`, `GET /products/:id`) & Task 3 (Menu UI)
- **Độ ưu tiên:** Cao (High) | **Story Points:** 3
- **Phạm vi áp dụng:** `apps/web` & `apps/api`

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** khách truy cập trang chủ, **When** ứng dụng đang gọi `GET /products`, **Then** hiển thị 6 khung chờ (skeleton loading).
2. **Given** API trả về danh sách đồ uống, **When** khách chọn tab danh mục (*Tất cả*, *Cà phê*, *Trà trái cây*, *Trà sữa & Matcha*) hoặc nhập từ khóa vào ô tìm kiếm, **Then** lưới món được lọc tức thì kèm hình ảnh, giá gốc và số ly tồn kho.
3. **Given** món có `stock <= 0`, **When** hiển thị trên menu, **Then** gắn nhãn *"Tạm hết món"* và vô hiệu hóa nút *"Tùy chọn"*.
4. **Given** gọi API `GET /products` thất bại, **When** trang hiển thị lỗi, **Then** hiện thông báo lỗi kèm nút *"Thử lại"* và không hiển thị nhầm trạng thái menu trống.

US-02: Tùy chỉnh thức uống (Size, Topping) và tính giá động

Mô tả (Story Narrative):

> Là một **Khách hàng (Customer)**,
> Tôi muốn **chọn kích cỡ (Size S/M/L), thêm topping và chọn số lượng trong giới hạn tồn kho**,
> Để **xem tổng giá chính xác trước khi đưa vào giỏ hàng**.

Thông tin bổ sung:

- **Task liên quan:** Task 4 (Chi tiết & tùy chọn sản phẩm)
- **Độ ưu tiên:** Cao (High) | **Story Points:** 5
- **Phạm vi áp dụng:** `apps/web` (`ProductModal`, `pricing.ts`)

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** khách bấm *"Tùy chọn"* ở một món còn hàng, **When** modal mở ra, **Then** hiển thị 3 kích cỡ (`S`: tiêu chuẩn, `M`: +5.000đ, `L`: +10.000đ), các topping (*Trân châu đen*, *Kem cheese*, *Thạch trái cây*: +5.000đ/loại) và bộ tăng giảm số lượng.
2. **Given** khách thay đổi Size, tick Topping hoặc đổi số lượng, **When** thao tác trên modal, **Then** tổng tiền thanh toán tự động cập nhật theo đúng công thức khớp với Backend.
3. **Given** số lượng món đã có trong giỏ đạt giới hạn tồn kho (`product.stock`), **When** khách mở lại modal, **Then** nút thêm bị khóa và thông báo đã đủ số ly còn lại.

US-03: Quản lý giỏ hàng (Cart) và lưu trữ trên trình duyệt

Mô tả (Story Narrative):

> Là một **Khách hàng (Customer)**,
> Tôi muốn **thêm, tăng/giảm số lượng, xóa món trong giỏ hàng và giữ lại giỏ khi tải lại trang**,
> Để **linh hoạt điều chỉnh đơn hàng trước khi thanh toán**.

Thông tin bổ sung:

- **Task liên quan:** Task 5 (Giỏ hàng & Kiểm thử tự động)
- **Độ ưu tiên:** Cao (High) | **Story Points:** 3
- **Phạm vi áp dụng:** `apps/web` (`CartProvider`, `CartDrawer`, `cart.ts`)

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** khách thêm món vào giỏ, **When** trùng món, Size và Topping đã có, **Then** hệ thống gộp dòng và cộng dồn số lượng (không vượt quá tồn kho), đồng thời cập nhật badge đếm tổng số ly trên Navbar.
2. **Given** khách mở *"Giỏ hàng"*, **When** bấm tăng/giảm số lượng hoặc xóa món, **Then** thành tiền từng món và tổng cộng được tính lại ngay lập tức.
3. **Given** giỏ hàng đang có món, **When** khách tải lại trang, **Then** dữ liệu giỏ hàng được khôi phục nguyên vẹn từ `localStorage` (`brewlite_cart`).

2.2 Epic 2: Đăng ký & Đăng nhập tài khoản (Task 7 – FR-04)

US-04: Đăng ký và đăng nhập bảo mật bằng JWT

Mô tả (Story Narrative):

> Là một **Khách hàng (Customer)**,
> Tôi muốn **đăng ký tài khoản mới và đăng nhập để nhận token JWT**,
> Để **đặt đơn hàng, theo dõi lịch sử đơn và tích lũy điểm thưởng cá nhân**.

Thông tin bổ sung:

- **Task liên quan:** Task 7 (`POST /auth/register`, `POST /auth/login`, `GET /auth/me`)
- **Độ ưu tiên:** Cao (High) | **Story Points:** 3
- **Phạm vi áp dụng:** `apps/api` (`AuthModule`, `JwtAuthGuard`) & `apps/web` (`AuthModal`)

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** khách gửi `POST /auth/register` với email chưa tồn tại và mật khẩu hợp lệ, **When** xử lý thành công, **Then** mật khẩu được băm bằng `bcrypt` (cost = 10) và lưu tài khoản vào PostgreSQL.
2. **Given** khách gửi `POST /auth/login` đúng thông tin, **When** xác thực thành công, **Then** hệ thống trả về `access_token` (JWT) cùng thông tin user (`id`, `email`, `loyaltyPoints`).
3. **Given** người dùng chưa đăng nhập (không có Bearer Token hợp lệ), **When** gọi `POST /orders`, `GET /orders/me` hoặc `GET /auth/me`, **Then** `JwtAuthGuard` trả về `401 Unauthorized`; trên giao diện khi bấm thanh toán sẽ mở modal đăng nhập.

2.3 Epic 3: Đặt hàng, Khuyến mãi & Thanh toán Không tiền mặt (Task 6, 8, 10)

US-05: Tạo đơn hàng, áp mã khuyến mãi và kiểm soát tồn kho đồng thời

Mô tả (Story Narrative):

> Là một **Khách hàng đã đăng nhập (Customer)**,
> Tôi muốn **gửi đơn hàng kèm mã khuyến mãi hợp lệ và được kiểm tra tồn kho chính xác**,
> Để **đơn hàng được ghi nhận ở trạng thái chờ thanh toán mà không xảy ra bán vượt tồn kho**.

Thông tin bổ sung:

- **Task liên quan:** Task 6 (`POST /orders`) & Task 10 (Optimistic Locking, Promotions)
- **Độ ưu tiên:** Khẩn cấp (Critical) | **Story Points:** 5
- **Phạm vi áp dụng:** `apps/api` (`OrdersService`, `PromotionsService`) & `CheckoutModal`

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** khách gửi `POST /orders` kèm JWT và danh sách món hợp lệ (`CreateOrderDto`), **When** `class-validator` xác thực đạt, **Then** Backend tự tính lại tổng tiền, trừ kho và lưu đơn hàng ở trạng thái `PENDING`.
2. **Given** khách nhập mã khuyến mãi (`BREW10` giảm 10%, `WELCOME20` giảm 20%), **When** kiểm tra qua `GET /promotions/validate` và tạo đơn, **Then** hóa đơn được giảm đúng phần trăm (hoặc báo `400 Bad Request` nếu mã không hợp lệ).
3. **Given** nhiều khách đặt cùng lúc một món sắp hết hàng, **When** thực thi trong Prisma Transaction với Optimistic Locking (`version` và `stock >= qty`), **Then** số lượng bán ra không bao giờ vượt quá tồn kho hiện có.

US-06: Thanh toán không tiền mặt (Mock Gateway, VNPay Sandbox) & Idempotency

Mô tả (Story Narrative):

> Là một **Khách hàng (Customer)**,
> Tôi muốn **thanh toán đơn hàng qua Ví điện tử/Thẻ mô phỏng hoặc cổng VNPay Sandbox và được bảo vệ khi lỗi hoặc bấm trùng**,
> Để **hoàn tất giao dịch an toàn, không bị trừ tiền hay cộng điểm hai lần**.

Thông tin bổ sung:

- **Task liên quan:** Task 8 (Cashless Payment) & Task 10 (Idempotency, Loyalty Points)
- **Độ ưu tiên:** Khẩn cấp (Critical) | **Story Points:** 5
- **Phạm vi áp dụng:** `apps/api` (`PaymentsService`) & `apps/web` (`CheckoutModal`, `MockPaymentGatewayModal`, `/vnpay-return`)

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** đơn ở trạng thái `PENDING`, **When** thanh toán thành công qua `POST /payments` (hoặc xác thực chữ ký HMAC-SHA512 trả về từ **VNPay Sandbox** tại `/payments/vnpay-verify`), **Then** đơn chuyển sang `PAID`, lưu bản ghi `Payment`, cộng điểm thưởng (10.000 VNĐ = 1 điểm) vào `LoyaltyTransaction` và xóa giỏ hàng.
2. **Given** giao dịch thất bại hoặc người dùng chọn mô phỏng lỗi/hủy trên cổng thanh toán, **When** API xử lý lỗi, **Then** đơn chuyển sang `PAYMENT_FAILED` và **giỏ hàng trên trình duyệt vẫn được giữ nguyên** để khách thử lại.
3. **Given** cùng một `idempotencyKey` được gửi lặp lại tới `POST /payments`, **When** hệ thống kiểm tra khóa duy nhất trong bảng `Payment`, **Then** trả lại kết quả cũ (`isIdempotentReplay: true`), không tạo thêm thanh toán và không cộng trùng điểm loyalty.

2.4 Epic 4: Xác nhận, Theo dõi lịch sử & Điều phối đơn hàng (Task 9, 10)

US-07: Xem xác nhận mã đơn và theo dõi tiến độ trong lịch sử đơn

Mô tả (Story Narrative):

> Là một **Khách hàng (Customer)**,
> Tôi muốn **xem màn hình xác nhận mã đơn ngay khi thanh toán xong và kiểm tra tiến độ đơn trong lịch sử**,
> Để **nắm rõ trạng thái đồ uống từ lúc chờ pha đến khi sẵn sàng nhận tại quầy**.

Thông tin bổ sung:

- **Task liên quan:** Task 9 (`GET /orders/me`, Order Confirmation & History)
- **Độ ưu tiên:** Trung bình (Medium) | **Story Points:** 3
- **Phạm vi áp dụng:** `apps/web` (`OrderConfirmationModal`, `OrderHistoryView`)

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** thanh toán thành công trên màn hình chính, **When** `OrderConfirmationModal` hiển thị, **Then** khách thấy mã đơn hàng (`#id`), trạng thái `PAID`, số điểm loyalty vừa nhận và lời mời tới quầy nhận nước.
2. **Given** khách đã đăng nhập mở tab *"Lịch sử đơn"*, **When** tải dữ liệu từ `GET /orders/me` (hoặc bấm nút *"Làm mới"*), **Then** hiển thị danh sách đơn từ mới nhất kèm thanh tiến trình 5 bước (`PENDING` → `PAID` → `PREPARING` → `READY` → `COMPLETED`).

US-08: Điều phối đơn hàng theo State Machine tại quầy Barista

Mô tả (Story Narrative):

> Là một **Nhân viên Pha chế (Barista / Quản lý quầy)**,
> Tôi muốn **xem danh sách đơn hàng tại màn hình quầy và cập nhật trạng thái xử lý theo đúng quy trình**,
> Để **đảm bảo đơn hàng chuyển trạng thái hợp lệ và đồng bộ cho khách hàng**.

Thông tin bổ sung:

- **Task liên quan:** Task 10 (Order State Machine: `PATCH /orders/:id/status`)
- **Độ ưu tiên:** Trung bình (Medium) | **Story Points:** 3
- **Phạm vi áp dụng:** `apps/api` (`order-state-machine.ts`) & `apps/web` (`/admin`)

Tiêu chí nghiệm thu (Acceptance Criteria):

1. **Given** Barista mở trang `/admin`, **When** hệ thống gọi `GET /orders` (tự động cập nhật mỗi 5 giây), **Then** hiển thị danh sách đơn và lọc theo các trạng thái (`PAID`, `PREPARING`, `READY`, `COMPLETED`, `PENDING`, `CANCELLED`).
2. **Given** đơn đang ở trạng thái hợp lệ, **When** Barista gọi `PATCH /orders/:id/status` theo luồng `PAID` → `PREPARING` → `READY` → `COMPLETED` (hoặc hủy sang `CANCELLED` từ `PENDING`/`PAID`/`PAYMENT_FAILED`), **Then** trạng thái đơn được cập nhật thành công.
3. **Given** có yêu cầu chuyển trạng thái sai quy tắc (nhảy cóc như `PENDING` → `READY` hoặc quay lui như `COMPLETED` → `PAID`), **When** hàm `assertTransition` kiểm tra, **Then** Backend từ chối và trả về lỗi `400 Bad Request`.